Transformer From Scratch - I

Muaaz Raza (*yourssharky@gmail.com*)

Abstract:

This document presents a from-scratch implementation of the Transformer architecture in NumPy. The goal of this exercise is not efficiency or novelty, but a rigorous understanding of the mathematics and mechanics underlying Transformers — building each component without relying on deep learning frameworks. This first part covers the architecture and forward pass: the design decisions, the mathematical formulation of each component, and their implementation. A second part covers the mathematical derivation and NumPy implementation of the backward pass, along with baseline training results on a character-level dataset.

Motivation:

The implementation relies solely on native Python and NumPy, at every stage from data loading through loss computation. The motivation for avoiding PyTorch or other deep learning frameworks is to understand what lies beneath their abstractions — to derive and implement every equation and gradient by hand, rather than delegate them to a library. The architecture follows the structure of Andrej Karpathy's nanoGPT (Karpathy, 2023), a minimal PyTorch implementation of the GPT-2 architecture, reimplemented here without any framework dependencies.

This is not intended as a novel or enhanced architecture, nor as a performance benchmark, but rather a step-by-step guide to implementing Transformers from scratch. This depth of understanding — of Transformer mechanics and the underlying mathematics — is intended as a foundation for implementing and validating claims from published research, working toward the eventual goal of producing original research.

Section 1: Architecture overview

1.1 Data Flow:

This architecture follows the general Transformer design introduced by Vaswani et al. (2017), consisting of stacked blocks of self-attention and feedforward sub-layers, adapted here as a decoder-only model per the design decisions discussed in Section 4.

The following diagram illustrates the model's end-to-end data flow, from raw input to output logits.

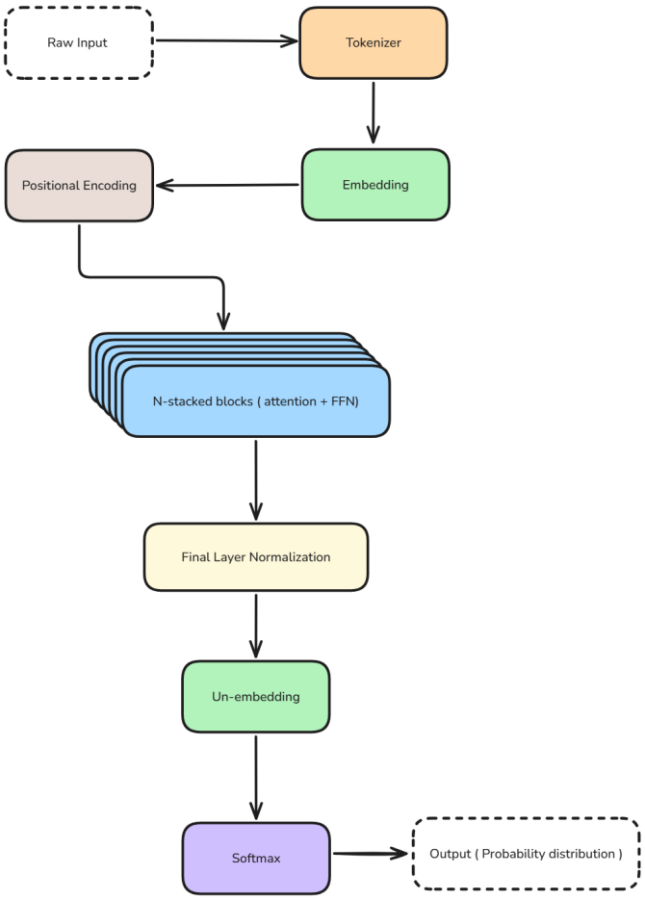


Figure 1 illustrates the model's forward data flow, from raw input through to output (probability distribution over the entire vocabulary).

Tokenization is handled via character-level encoding: each unique character in the training corpus is mapped to an integer ID, forming the model's vocabulary. This step is treated as a preprocessing detail and is not elaborated further in this document.

1.2 Design Decisions:

The following design decisions shape this implementation:

1. **Pre-LayerNorm**: Layer Normalization is applied to input before feeding them into attention/FFN blocks.
2. **Causal masking:** Attention restricted to the current and preceding positions, enforcing the decoder-only, autoregressive nature of the model (each position can only attend to itself and earlier positions, so generation proceeds one token at a time based on prior context).

3. **Separate per-head weight matrices:** each attention head has its own Wq/Wk/Wv, rather than a single fused projection split across heads.
4. **Final LayerNorm before unembedding**: One additional normalization applied after the last stacked block, before the output projection.

1.3 Component Inventory:

Section 2 — Mathematical Preliminaries, used at multiple points throughout the architecture rather than appearing once:

- 2.1 Softmax
- 2.2 Layer Normalization
- 2.3 Residual Connection
- 2.4 GELU

Section 3 — Component Walkthrough, covering the model's actual forward pass in order, from raw tokens to final output, referencing the building blocks above as needed:

- 3.1 Token Embedding
- 3.2 Positional Encoding
- 3.3 Self-Attention (Causal Masking, Single-Head, Multi-Head)
- 3.4 Feedforward Block (FFN)
- 3.5 Output Projection / Unembedding

1.4 Notation Table:

The following table lists the scalar hyperparameters used throughout this implementation. Symbols for intermediate tensors and matrices are defined individually within each component's own section, in its corresponding Shape table.

Symbol	**Meaning**
seq_len	Length of current input sequence
vocab_size	Total number of unique tokens (characters in our case) in the vocabulary
d_model	Embedding dimension size
d_k	Key/query weight matrices dimension size
d_v	Value weight matrix dimension size
ffn_wd	FFN Hidden layer dimension
layers	Number of layers
heads	Number of attention heads

Section 2: Mathematical Preliminaries (prerequisites, order doesn't imply pipeline sequence)

2.1 Softmax function:

Softmax converts a vector of real-valued scores into a probability distribution — each output value is non-negative and the full vector sums to 1. For a vector x = [x_1, ..., x_n], the softmax function is defined as:

$$softmax(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

where the numerator exponentiates the i-th element, and the denominator sums the exponentials of all elements in the vector, ensuring the outputs sum to 1.

NumPy Implementation:

```
def softmax(x):
    x_shifted = x - np.max (x, axis=1, keepdims=True)
    return np.exp(x_shifted)/(np.sum(np.exp(x_shifted), axis=1, keepdims=True))
```

In practice, the input is shifted by subtracting the row-wise maximum before exponentiation, to prevent numerical overflow when x contains large values. This shift does not alter the output, since softmax is invariant to adding or subtracting a constant from all inputs — the constant cancels between the numerator and denominator.

2.2 Layer Normalization:

Intuition:

After each attention or FFN sub-layer, the scale of the output can become extreme — either very close to zero (vanishing) or very large (exploding). This effect can accumulate across multiple sequential layers, resulting in unstable activation values. In the backward pass, these extreme values can produce correspondingly extreme gradients, which can cause some neurons or pathways to stop learning entirely. Mitigating this without normalization would require a very small, carefully chosen learning rate, which makes training drastically slower and less efficient.

Normalization addresses this directly: the input to each sub-layer is rescaled to have a mean of approximately 0 and a standard deviation of approximately 1, keeping activation values within a stable, consistent range regardless of depth. In addition, two learnable parameters are introduced, allowing the model to scale and shift the normalized output — so that normalization constrains the scale of activations without permanently removing the model's ability to represent values at other scales, if that proves useful for learning.

Math:

Layer Normalization is applied to an input of shape (seq_len, d_model). For each row — corresponding to a single token's position in the sequence — the mean and variance are computed across the d_model dimension, and that row is normalized using its own mean and variance, independently of every other row.

$$LayerNorm(x, \mu, \sigma^2) = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} \odot \gamma + \beta$$

where:

$\mu = mean = \frac{\sum_{i=1}^{n} x_i}{d_{model}}$

$\sigma^2 = variance = \frac{\sum_{i=1}^{n} (x_i - \mu)^2}{d_{model}}$

γ = trainable parameter for scaling input values, of size (1, d_model)

β = trainable parameter for shifting input values, of size (1, d_model)

ε = negligible value to avoid division by zero

Note that each block has their distinct γ, β (i.e attention block , FFN block etc)

NumPy Implementation:

```
def LayerNorm(x,epsilon,gamma,beta):
   mean = np.mean(x,axis=1,keepdims=True)
   var = np.var(x,axis=1,keepdims=True)
   return ((x-mean)/(var+epsilon)**0.5)*gamma + beta
```

2.3 Residual Connections:

Intuition:

As neural networks grow deeper, a specific optimization barrier emerges: network degradation. Beyond a certain depth, adding more layers to a plain network causes training error to *increase*, even when gradients remain numerically stable — a counterintuitive result, since a deeper model should, in principle, perform at least as well as a shallower one by having its extra layers learn an identity transformation. He et al. (2015) identifies this as the core motivation for residual learning: in practice, standard non-linear layers find it difficult to learn a precise identity mapping, making this theoretical guarantee hard to achieve through direct optimization alone.

Residual connections (or skip connections), as proposed by He et al. (2015), resolve this by reframing the learning task. Instead of forcing a block to learn an entirely new representation from scratch, the

architecture explicitly passes the input forward through an unimpeded path. The block itself is only tasked with learning the incremental refinement — the residual — needed to improve upon the existing representation.

Math:

For a given input x, let $H(x)$ represent the desired underlying mapping of a Transformer block (such as Multi-Head Attention or a Feed-Forward Network). We re-parameterize $H(x)$ by defining a residual mapping $F(x)$:

$$F(x) = H(x) - x$$

Rearranging this gives the standard residual forward pass:

$$y = F(x) + x$$

If the optimal transformation for a given layer is simply the identity, the optimizer can trivially drive the weights of $F(x)$ toward zero ($F(x) \to 0$), making the default behavior of any layer an identity pass-through ($y = x$).

2.4 GELU

Intuition

GELU (Gaussian Error Linear Unit) is the non-linear activation function used within the Feedforward block. Unlike ReLU, which sharply zeroes out all negative inputs, GELU weights its output by the input's position in a standard Gaussian distribution — producing a smooth, non-monotonic curve that scales inputs down gradually near zero rather than cutting them off abruptly. This smoothness gives the model a small amount of stochastic-like regularity in its non-linearity, which has been shown empirically to improve training in Transformer architectures.

Math

$$\text{GELU}(x) = x \cdot \Phi(x)$$

where Φ(x) is the standard Gaussian cumulative distribution function — the probability that a standard normal random variable is less than or equal to x. In practice, Φ(x) is expensive to compute exactly, so implementations commonly use a fast, close approximation:

$$\text{GELU}(\text{x}) \approx 0.5\text{x}\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}(\text{x} + 0.044715\text{x}^3)\right]\right)$$

NumPy Implementation:

```
def gelu(x):
```

```
    inner = ((2 / np.pi) ** 0.5) * (x + 0.044715 * (x ** 3))
    return 0.5 * x * (1 + np.tanh(inner))
```

Section 3: Component Walkthrough

3.1 Token Embeddings:

Intuition:

An encoded token (a single integer, per the character-level scheme described above) does not carry enough information to represent a token's semantics — for example, whether it relates to a masculine or feminine class, or a singular or plural form. To address this limitation, each token is mapped into a high-dimensional vector space, with sufficient dimensions to capture such semantic properties.

The embedding vectors are not fixed or hand-designed; they are **learnable parameters**, initialized randomly and refined during training via gradient descent. The resulting vector space is not structured along interpretable axes — no single dimension corresponds directly to a concept such as gender or number. Rather, semantic relationships emerge collectively across the dimensions as a consequence of training on the data.

Math:

The embedding operation is a lookup, formalized as follows:

$$E_i = W_{emb}[T_i]$$

Where:

T = the token ID vector for the input sequence, of shape (seq_len,), containing the encoded token at each position

T_i = Token ID at position ‘i’ in T

W_{emb} = embedding matrix, of shape (vocab_size, d_model)

E_i = resulting embedding vector for position i, of shape (d_model,)

When lookup operation is applied across a full sequence, it produces an embedding matrix E

$$E = \begin{bmatrix} E_1 \\ \vdots \\ E_n \end{bmatrix}$$

where **n** = seq_len and **E** is of shape (seq_len, d_model)

Shape:

Symbol	Description	Shape
T	Token ID vector for input sequence	(seq_len,)
Wemb	Embedding matrix for entire vocabulary	(vocab_size, d_model)
Ei	Embedding vector at position (row) i	(d_model,)
E	Embedding matrix for full sequence	(seq_len, d_model)

Table 3.1 summarizes the shapes of all tensors introduced in this section. Note that the lookup operation transforms a sequence of scalar token IDs into a sequence of dense vectors — increasing the per-position representation from a single integer to a d_model-dimensional vector, while preserving the sequence length.

NumPy Implementation:

```
E = np.stack([Ei := Wemb for ti in T])
```

3.2 Positional Encoding:

Intuition:

Within the attention mechanism, every token attends to every other token in parallel, with no inherent notion of position — the model has no built-in way to know which token came before another. This creates a problem for tasks that depend on order or relative position: consider the two sentences 'The dog bit the man' and 'The man bit the dog.' Both contain the exact same set of tokens — only their order differs — yet they describe entirely different events. Since attention operates on tokens without inherent regard to sequence position, the model would treat both sentences as identical unless positional information is explicitly provided.

Math:

$$pe(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$pe(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

where:

pos = the position index of the token within the sequence (0 to seq_len − 1)

i = the pair index (0, 1, ..., d_model/2 − 1); 2i and 2i+1 are the corresponding even and odd dimensions of the encoding vector.

d_model = embedding dimension size

Here positional encoding function (pe) is applied to every single token's embedding vector in the input sequence:

$$PE = \begin{bmatrix} \text{PE}_{pos=1} \\ \vdots \\ PE_{pos=seq_\text{len}} \end{bmatrix}$$

The positional encoding matrix **PE** is added element-wise to the token embedding matrix **E**. The resulting vector for each position encodes both what the token is and where it occurs — though these two signals are superimposed within the same vector, not stored as separate components:

$$Xp = E + PE$$

Where Xp = Updated embedding matrix containing both semantic and positional information, of shape (seq_len, d_model)

Characteristics of sinusoidal positional encodings:

The properties discussed below draw on the sinusoidal encoding formulation as commonly presented in the literature (Shubham, 2025), in addition to the properties derived directly from the formula above.

Boundedness: A periodic function was chosen over a simpler alternative, such as the raw position index, because raw position values are unbounded — they grow linearly with sequence length, producing large magnitudes for later positions that are inconsistent in scale with the token embedding values they are added to. Sine and cosine are bounded within [-1, 1] regardless of position or sequence length, keeping the positional signal at a consistent scale.

Relative-distance recoverability: The use of sine and cosine across dimension pairs allows the encoding at position *pos + k* to be expressed as a linear function of the encoding at position *pos*, for any fixed offset *k*. As a result, the dot product between two positional encodings reflects the distance between the corresponding positions, allowing attention to implicitly account for relative position.

Multi-resolution frequency structure: The frequency term, $10000^{(2i/\mathrm{dmodel})}$, grows with the dimension pair index *i*, so frequency decreases as *i* increases. As a result, lower-indexed dimensions oscillate rapidly across positions, capturing fine-grained, local positional detail, while higher-indexed dimensions oscillate slowly, capturing coarse-grained, global positional structure.

Extrapolation to unseen sequence lengths: Because the encoding is computed from a fixed formula rather than learned, it can be evaluated for any position, including positions beyond the maximum sequence length seen during training. This allows the model to generalize to longer sequences than it was trained on — unlike a learned positional embedding table, which is fixed in size and undefined beyond its trained range.

Shape:

Symbol	Description	Shape
PE	Positional encoding of n tokens in input sequence	(seq_len, d_model)
Xp	Updated Embedding matrix with positional information attached	(seq_len, d_model)

Table 3.2 summarizes the shapes of all tensors introduced in this section.

NumPy Implementation:

```
def positional_encoding(E):
        seq_len = E.shape[0]
        d_model = E.shape[1]
        PE = np.zeros((seq_len, d_model))

        #  numerator for every position
        positions = np.arange(seq_len)[: , np.newaxis]

        #  denominator for every i
```

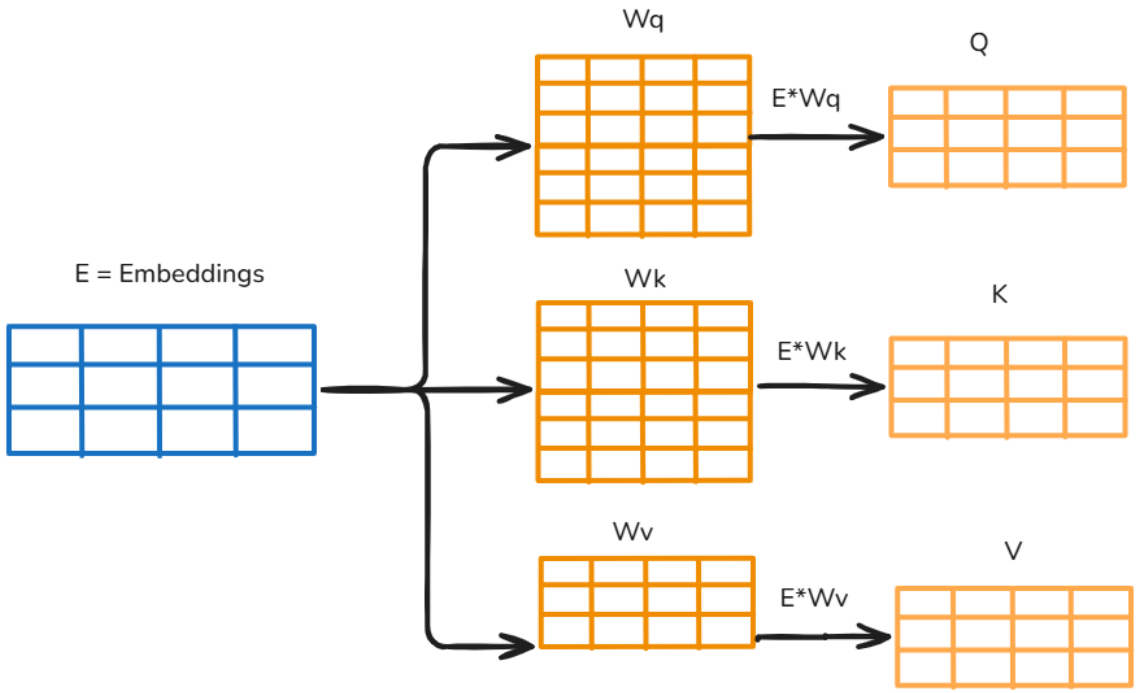
```
        divisor = 10000**(np.arange(0, d_model, 2)/d_model)

        #  applying sine to even positions and cos to odd ones
        PE[:, 0::2] = np.sin(positions/divisor)
        PE[:, 1::2] = np.cos(positions/divisor)

        Xp = E + PE
        return Xp
```

3.3 Self Attention (Causal masking, Single-Head & Multi-Head):

Components of self attention block:



Query matrix (Q): The query matrix is obtained by multiplying the embedding matrix by Wq, a traina

ble weight matrix. It can be thought of as the model's way of asking, "how relevant is this word in the context of the entire sequence?"

Key matrix (K): The key matrix is obtained by multiplying the embedding matrix by Wk, a trainable weight matrix. The keys serve as reference points that each query vector is compared against — they allow the model to determine how closely related each word in the sequence is to the word currently under focus.

Value matrix (V): The value matrix is obtained by multiplying the embedding matrix by Wv, a trainable weight matrix. Each word in the sequence has an associated value vector, which holds its contextual content. Once the model has determined how much attention to assign to each word — based on the comparison between queries and keys — it uses the value vectors to construct a weighted representation of the context.

3.3.1 Causal Masking (M):

Since the model is intended to be autoregressive, each position's output should be usable to predict the next token — not just the output at the final position. This is achieved by restricting each token to attend only to itself and preceding tokens, so that the prediction at every position depends only on current and prior context, never on future tokens.

This restriction is implemented via a mask matrix, structured so that entries above the diagonal (upper triangular, representing future positions) are set to -∞, and entries on or below the diagonal (representing current and prior positions) are set to 0. Since the mask is added directly to the scaled attention scores before softmax is applied, positions set to 0 leave the corresponding score unchanged, while positions set to -∞ drive the score to negative infinity. After softmax, these masked positions receive a probability of approximately zero, effectively preventing the model from attending to future tokens.

Formally, for a mask matrix M of shape (seq_len, seq_len):

M[i,j] = 0 if j ≤ i, and M[i,j] = -∞ if j > i

where *i* is the query position (row) and *j* is the key position (column)

$$M = \begin{bmatrix} 0 & -\infty & \cdots & -\infty \\ 0 & 0 & -\infty & \cdots \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}$$

NumPy Implementation:

```
mask = np.tril(np.ones((block_size)))
mask[mask == 0] = -np.inf
mask[mask == 1] = 0
```

3.3.2 Single Head Attention:

Intuition:

Although raw tokens are mapped to embedding vectors with the goal of providing enough dimensions to capture a token's semantics, a critical limitation of the embedding space is that it maps each unique token to a single, fixed vector — regardless of context. This makes it unreliable for capturing context-dependent meaning. For example, the word "light" can refer to something of low weight or to illumination from a light source; this ambiguity can only be resolved by considering the surrounding context. **Self-attention** addresses this limitation by allowing the model to update each token's embedding based on its surrounding tokens, producing what are called contextual embeddings. This allows the model to capture more nuanced, context-dependent meaning for each token, and to disambiguate the meaning of the sentence as a whole.

A contextual embedding is a token's vector representation after it has been updated to reflect the specific context in which the token appears, as opposed to the fixed, context-independent embedding produced by the embedding layer alone.

Math:

Xin denotes the input to a given block at any layer: equal to *Xp* for the first layer, and equal to the previous layer's *Xfn* for every subsequent layer.

Xin is first normalized using layer-normalization to produce *Xn1* as follows:

$$Xn1 = LayerNorm(Xin)$$

Xn1 is multiplied by Wq, Wk and Wv separately to produce K, Q and V matrices.

Multiplication is Applied to the full input sequence; the same projections are computed for every position simultaneously via matrix multiplication:

$$Q = Xn1.Wq$$

$$K = Xn1.Wk$$

$$V = Xn1.Wv$$

Attention weights are computed by scaling, adding causal mask and applying softmax to the dot product of Q and K:

$$A(Q,K) = Softmax(\frac{Q.K^T}{\sqrt{d_k}} + M)$$

The resulting attention weights are then applied to V to produce attention output:

$$Attv(A,V) = A.V$$

Finally, applying the residual connection to produce final output of the attention block:

$$Xat = Xin + Attv$$

where:

Wq = trainable query weight matrix, of dimension (d_model, d_k)

Wk = trainable key weight matrix, of dimension (d_model, d_k)

Wv = trainable value weight matrix, of dimension (d_model, d_v)

The scores QK^{T} are divided by $\sqrt{d_k}$ before softmax is applied. Without this scaling, the variance of the dot products grows with *d_k*, pushing many values into the extreme range of the softmax function. This causes the softmax output to become sharply peaked, with gradients vanishing almost everywhere except at the maximum — which slows or destabilizes learning. Dividing by $\sqrt{d_k}$keeps the variance of the scaled scores approximately constant regardless of *d_k*, avoiding this saturation.

Additional information: In single-head attention*, d_v* is typically set equal to *d_model*, so the output dimension matches (seq_len, d_model) directly, with no additional projection required. In multi-head attention, each head instead uses a smaller *d_v* (typically d_model / num_heads), and the outputs of all heads are concatenated and passed through a learned projection matrix Wp to produce the final output of shape (seq_len, d_model). This projection is not merely a dimensional adjustment — even when concatenation alone would already produce the correct output dimension, Wp serves to combine information across heads through a trainable transformation, rather than leaving each head's contribution isolated in a separate slice of the output vector.

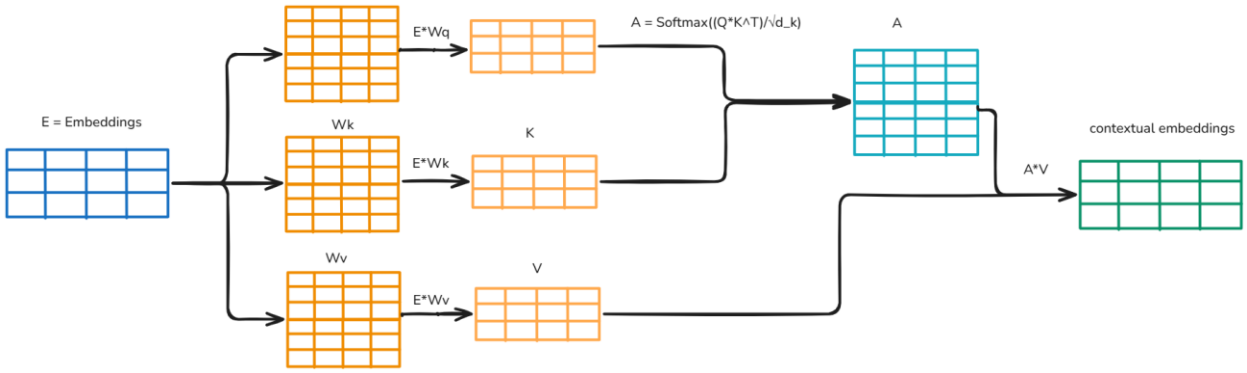


Figure 3.3.2 illustrates the computation of single-head self-attention, from the input embeddings through to the resulting contextual embeddings

Shape:

Symbol	Description	Shape
Xin	Input matrix to the attention block	(seq_len, d_model)

Xn1	Normalized input using layer-norm	(seq_len,d_model)
Wq, Wk (single head)	Query and Key weight matrices	(d_model, d_k)
Wv (single head)	Value weight matrix	(d_model, d_v)
Q, K	Query and Key matrices	(seq_len, d_k)
V	Value matrix	(seq_len, d_v)
A	Intermediate attention variable before multiplying with V	(seq_len, seq_len)
Attv	Final Attention output	(seq_len, d_model)
Xat	Final attention block output (Contextual Embeddings)	(seq_len, d_model)

Table 3.3.2 summarizes the shapes of all tensors introduced in this section.

NumPy Implementation:

```
def attention(self, Xin, layer):
     Xn1 = LayerNorm(Xin)
     V = Xn1 @ Wv
     Q = Xn1 @ Wq
     K = Xn1 @ Wk

     A = softmax( (Q @ K.T / ((d_k)**0.5)) + mask)
     attv = A @ V
     Xat = Xin + attv
     return Xat
```

3.3.3 Multi-Head Attention:

Intuition:

A limitation of single-head attention is that a single set of Q, K, and V projections can only learn one criterion for relating tokens to each other — for example, similarity based on positional proximity. Other relevant factors, such as grammatical relationships between words, may go uncaptured, since the single head must compress all such relationships into one shared similarity score. Multi-head attention addresses this by using multiple independent sets of Q, K, and V projections — each head computes its own attention independently and in parallel, allowing different heads to potentially specialize in different relational aspects of the tokens. The outputs of all heads are concatenated into a single matrix, which is then projected through a learned matrix, Wp, allowing the model to learn how to combine and weight the contributions of each head.

Components of multi-head attention:

Query, Key, and Value matrices: These operate the same as in single-head attention, except that n heads each perform these operations on their own separate Q, K, and V projections, in parallel.

Projection matrix (Wp): Once each attention head has computed its output, the outputs are concatenated along the last dimension, yielding a matrix of shape (seq_len, n_heads × d_v). This concatenation is static — it simply places each head's output side by side, with no learned interaction between them. The projection matrix Wp transforms this static concatenation into a dynamically weighted combination, learned during training, producing a final output of shape (seq_len, d_model). Although n_heads × d_v may already equal d_model, Wp is still necessary: it allows the model to learn how to selectively combine and weight the relevant features from each head's output, rather than leaving them as an unweighted concatenation.

Math:

Xin denotes the input to a given block at any layer: equal to *Xp* for the first layer, and equal to the previous layer's *Xfn* for every subsequent layer.

Xin is first normalized using layer-normalization to produce *Xn1* as follows:

$$Xn1 = LayerNorm(Xin)$$

Xn1 is multiplied by Wq_i, Wk_i, and Wv_i (the query, key, and value weight matrices for the i-th head — to produce Q_i, K_i, and V_i:

$$Q_i = Xn1.Wqi$$

$$K_i = Xn1.Wki$$

$$V_i = Xn1.Wvi$$

For each head, attention weights are computed by scaling the dot product of Qi and Ki, adding the causal mask, and applying softmax — exactly as in single-head attention:

$$\mathrm{A}(Q,\mathrm{K})_{\,i} = Softmax\left(\frac{Q_i.{K_i}^{\mathrm{T}}}{\sqrt{d_k}} + M\right)$$

The resulting attention weights are then applied to V_i to produce each head's output:

$$Att(A,V)_i = A_i.V_i$$

Since each head uses its own independent set of Wq_i, Wk_i, and Wv_i the heads compute their attention outputs in parallel and independently of one another — each potentially attending to different relational aspects of the input, as discussed above.

The outputs of all heads are concatenated along the last dimension into a single matrix:

$$AttCon = [Att_i \mid \cdots \mid Att_{n_head}]$$

Each Att_i is of shape (seq_len, d_v); concatenating n_head such matrices produces Att_conc of shape (seq_len, n_head × d_v), which equals (seq_len, d_model) when d_v = d_model / num_heads, per the earlier note on multi-head dimension sizing.

Finally, Att_conc is projected through the learned matrix Wp, allowing the model to combine information across heads:

$$ProjectedAtt = AttCon.Wp$$

Finally, applying the residual connection to produce final output of the attention block:

$$Xat = Xin + ProjectedAtt$$

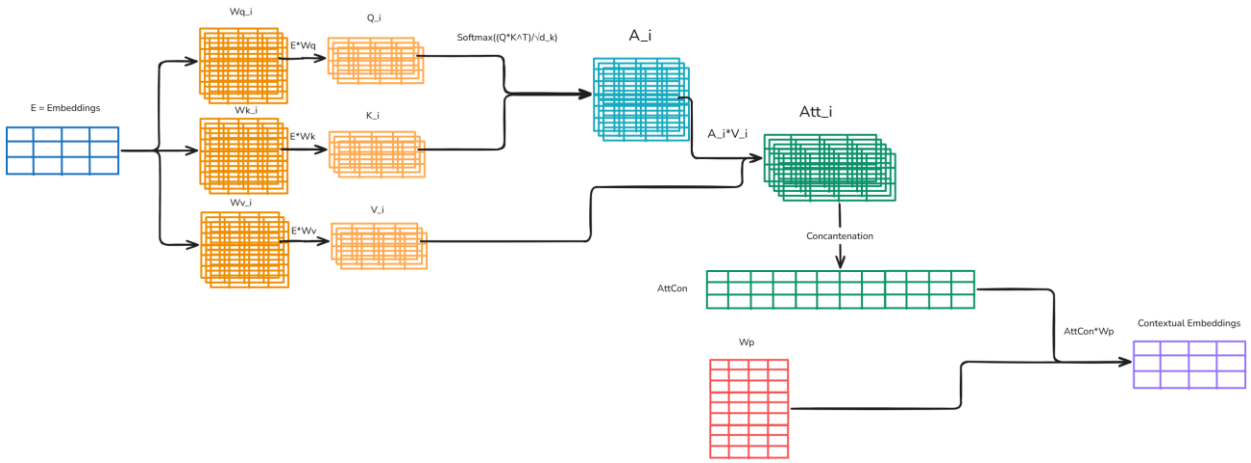


Figure 3.3.3 illustrates the computation of multi-head self-attention, from the input embeddings through to the resulting contextual embeddings

Shape:

Symbol	Description	Shape
Xin	Input matrix to the attention block	(seq_len, d_model)
Xn1	Normalized input using layer-norm	(seq_len,d_model)
Wq, Wk	Query and Key weight matrices	(n_heads, d_model, d_k)

Wv	Value matrix	(n_heads, d_model, d_v)
Q, K	Query and Key matrices	(n_heads, seq_len, d_k)
V	Value matrix	(n_heads, seq_len, d_v)
A	Intermediate variable before Value matrix multiplication	(n_heads, seq_len, seq_len)
Att	All attention head results	(n_heads, seq_len, d_v)
AttCon	Concatenated attention results	(seq_len, n_heads*d_v)
Wp	Attention Projection matrix	(n_heads*d_v, d_model)
projectedAtt	Multi head attention projected result	(seq_len, d_model)
Xat	Final multi-head attention block result (Contextual embeddings)	(seq_len, d_model)

Table 3.3.3 summarizes the shapes of all tensors introduced in this section.

NumPy Implementation:

```
def multiHeadattention(self, Xin, layer):
      Xn1 = LayerNorm(Xin)

      V = Xn1 @ Wv
      Q = Xn1 @ Wq
      K = Xn1 @ Wk

      A = softmax( (Q @ np.transpose(K,axes=(0,2,1)) / ((d_k)**0.5)) + mask)

      att = A @ V
      seq_len = att.shape[1]
      AttCon = np.reshape(np.transpose (att, axes= (1,0,2)), (seq_len, -1))

      projectedAtt = AttCon @ Wp

      Xat = Xin + projectedAtt

      return Xat
```

3.4 FFN Block

Intuition:

Feed-forward layers can feel like an opaque non-linear transformation with little explanation for what they're actually doing — but recent interpretability work (Geva et al., 2021) offers a more intuitive picture: think of the FFN as a large bank of pattern-matchers. Each unit inside the hidden layer has learned to recognize a particular kind of input — a phrase, a topic, a grammatical cue — and has, separately, learned an opinion about what word tends to come next whenever that pattern shows up. When a token's representation strongly resembles one of these learned patterns, that unit "lights up," and its associated opinion gets blended into the output, roughly in proportion to how strongly it recognized the pattern. Many units can light up at once, each contributing its own small vote — unlike attention, where the votes are forced to compete for a limited share; here, several strong matches can all count fully at the same time.

The interesting part is that these two halves — "what pattern do I detect" and "what do I predict comes next" — are never explicitly programmed to relate to each other. They emerge together, over the course of training, simply because they're always used as a pair: whenever a unit's pattern-detector fires, its paired "opinion" is the one that gets nudged by the training signal. Over many examples, this naturally aligns the two, without either half ever being told what the other is doing.

Math:

Layernorm is applied to input matrix Xat:

$$Xn2 = LayerNorm(Xat)$$

where Xat is the residual-connected output of the preceding attention block, of shape (seq_len, d_model).

The first FFN layer transformation is applied to Xn2 as follows:

$$Z_0 = Xn2.\mathrm{W}_0 + B_0$$

$$A_0 = GELU(Z_0)$$

The hidden layer output is then propagated to the final layer of the FFN:

$$Z_1 = A_0.W_1 + B_1$$

Applying the residual connection to the original input produces the output of the FFN block:

$$Xfn = Xat + Z_1$$

where:

GELU is the non-linear activation function.

W0 and W1 are the trainable weight matrices (referred to as keys and values, respectively, in the Intuition section above), of dimension (d_model, ffn_wd) and (ffn_wd, d_model).

B0 and B1 are the bias vectors, of shape (1, ffn_wd) and (1, d_model) respectively.

Xfn is the output of the FFN block, of shape (seq_len, d_model), matching Xat.

Shape:

Symbol	Description	Shape
Xat	Input matrix to the ffn block	(seq_len, d_model)
Xn2	Normalized input (Xat -- coming from attention block) using layer-norm.	(seq_len,d_model)
A0	1st FFN's hidden layer's output	(seq_len,ffn_wd)
W0	1st Hidden layer trainable weight matrices	(d_model,ffn_wd)
Z1	2nd linear layer output	(seq_len,d_model)
W1	2nd Hidden layer trainable weight matrices	(ffn_wd, d_model)
B0	1st Hidden Layer's bias vector	(1, ffn_wd)
B1	2nd Hidden Layer's bias vector	(1, d_model)
Xfn	Final output of FFN's layer	(seq_len,d_model)

Table 3.4 summarizes the shapes of all tensors introduced in this section.

NumPy Implementation:

```
def feedforwardlayer(self, Xat, layer):

        Xn2 = layer_norm(Xat,epsilon,gamma,beta)

        A0 = gelu(Xn2  @ self.params.ffn.W0[layer] + self.params.ffn.B0[layer])

        Z1 = A0 @ self.params.ffn.W1[layer] + self.params.ffn.B1[layer]

        Xfn = Z1 + Xat

        return Xfn
```

3.5 Output Projection/UnEmbeddings

Intuition:

Although computation throughout the network has taken place in the d_model-dimensional space, the final output is required as a probability distribution over the entire vocabulary. Producing this requires

one last transformation, projecting from d_model to vocabulary size via the unembedding matrix, Wu. Since this implementation uses pre-LN, the residual stream itself is never normalized during the forward pass — each block only normalizes a copy of it before feeding it into a sub-layer, while the stream itself continues to accumulate un-normalized. A final LayerNorm is therefore required before this last projection, to stabilize the accumulated stream's scale before it is projected to logits.

The resulting logits are then converted into a probability distribution over the vocabulary via softmax. While many implementations fold this step into the loss function during training and apply it explicitly only at inference time, this implementation computes softmax unconditionally on every forward pass, for both training and generation — favoring a single, consistent computation path over the minor efficiency gain of fusing softmax into the loss.

Math:

Layer-norm is applied to input matrix Xfn:

$$Xlf = Layernorm(Xfn)$$

Now final projection is applied to the normalized input Xlf:

$$Xv = Xlf.Wu$$

Softmax function is applied to convert logits (Xv) into probability distribution and the final output of the entire transformer's forward pass:

$$Xf = Softmax(Xv)$$

where:

Xlf = normalized output after applying the final LayerNorm, of shape (seq_len, d_model)

Wu = Unembedding/Projection trainable matrix, of shape (d_model, vocab_size)

Xv = logits produced by applying projection transformation, of shape (seq_len, vocab_size)

Xf = Probablity distribution over the entire vocabulary (all characters), of shape (seq_len, vocab_size)

Shape:

Symbol	Description	Shape
Xfn	Input matrix to the final block	(seq_len, d_model)
Xlf	Normalized input (Xfn —coming from attention block) using layer-norm.	(seq_len,d_model)
Wu	Unembedding trainable weight matrix	(d_model,vocab_size)
Xv	Logits	(seq_len, vocab_size)
Xf	Final probability distribution	(seq_len, vocab_size)

Table 3.5 summarizes the shapes of all tensors introduced in this section.

NumPy Implementation:

```
def final_step(self, Xfn):

        Xlf = layer_norm(Xfn, epsilon,gamma, beta )
        Xv = Xlf @ self.params.final.Wu
        Xf = softmax(Xv)

        return Xf
```

Section 4: Design Decisions & Trade-offs

4.1 Pre-Layer Normalization:

BatchNorm vs. LayerNorm:

BatchNorm computes mean and variance per feature channel by pooling across the batch (and, in CNNs, spatial positions). This is justified in CNNs by convolution's translation-invariance — the same filter applies everywhere, so a channel's statistics are comparable across positions.

This does not hold for Transformer sequences. Under causal masking, position 1 sees only itself while position 50 sees fifty tokens of context, and positional encoding adds a distinct value to each position. With no structural symmetry to justify treating positions as statistically comparable, pooling across positions and across unrelated sequences in a batch mixes representations that aren't comparable.

This also breaks autoregressive generation: training computes statistics over complete sequences, while generation proceeds one token at a time, often at batch size one — leaving no meaningful batch variance to compute and forcing reliance on training-time running statistics that are already compromised by the mismatch mentioned above.

LayerNorm avoids this entirely by computing mean and variance across the feature dimension (d_model) for a single token, independent of batch and position. Its computation is identical at training and inference time, regardless of batch size or sequence length — making it the appropriate choice here.

Having established LayerNorm as the appropriate normalization technique, a second design decision remains: where within each block the normalization step should be placed.

Post-LN vs pre-LN:

Post-LN refers to the placement of the normalization step after the residual connection — that is, after the module's output has already been added to the original input. The module transformation itself is therefore applied to the raw, un-normalized input. This is written mathematically as follows:

$$Y = LayerNorm\big(X_0 + Module(X_0)\big)$$

It is well established that, by stabilizing activation variance at a constant scale, Post-LN prevents activations from growing unbounded. However, because gradients flowing back through the residual connection must also pass through this normalization step, Post-LN produces large gradients near the output layer at initialization — and this magnitude does not naturally decay as gradients propagate toward earlier layers. Several studies (Xiong et al., 2020; Kedia et al., 2024) show this can degrade gradient flow in deeper networks, causing instability that typically requires a carefully tuned learning-rate warmup schedule to avoid divergence, and can otherwise lead to vanishing gradients and slower convergence.

Pre-LN refers to the placement of the normalization step before the module transformation (attention or FFN), so that the transformation is applied to the normalized input rather than the raw one. The transformed output is then added to the original, raw input via a residual connection. This is written mathematically as follows:

$$Y = X_0 + Module\big(LayerNorm(X_0)\big)$$

where Y is the sum of the module's output and the original input X0. Note that normalization is applied to the input before the module function, and the residual connection carries the raw, un-normalized input forward. Additionally, a final normalization is applied after the last stacked block. Unlike Post-LN, gradients in Pre-LN are freer to flow directly to earlier layers via the residual path, which helps stabilize early training. Nonetheless, even Pre-LN is not immune to instability in deep Transformers: the continuous, un-normalized residual "highway" can allow magnitude spikes produced by any individual module to accumulate unchecked across layers. This phenomenon may be related to the massive activations observed in trained models.

Why Pre-LN is used:
At the small scale used here — a shallow network trained on a character-level dataset — Pre-LN was chosen over Post-LN because its main drawback, exponential activation variance growth across depth, becomes significant primarily in very deep, large-scale models (typically cited beyond tens or hundreds of billions of parameters) and is negligible at this depth. Post-LN's drawback, by contrast, is an immediate practical obstacle at any scale: unstable training without a carefully tuned warmup schedule. Pre-LN therefore offers simpler, more stable training dynamics here without incurring the cost that limits it at scale.

4.2 Causal masking / Decoder-Only Design:

The Alternatives

Three broad architectural families exist, each tied to a different training objective: encoder-only models (e.g., BERT), trained with bidirectional masked language modeling, suited to understanding tasks such as classification; encoder-decoder models, which process an input into an intermediate representation

before generating an output, suited to sequence-to-sequence tasks such as translation; and decoder-only models, trained on causal language modeling, suited to open-ended generation.

The Mechanism

Causal masking restricts each position's attention to itself and preceding positions, ensuring a token's prediction depends only on prior context (Radford et al., 2018). This enforces the autoregressive property: generation proceeds one token at a time, with each generated token fed back in as context for the next, under a constraint identical to the one used during training.

The Trade-off

The choice of architecture is task-dependent, not universally settled. Raffel et al. (2020) found encoder-decoder architectures outperformed decoder-only models for transfer learning on certain tasks. However, a broader empirical comparison found causal decoder-only models performed best immediately after pretraining, and that all state-of-the-art language models beyond 100 billion parameters were causal decoder-only (Wang et al., 2022).

Why causal masking is used:

This implementation is built around open-ended, autoregressive text generation — the task decoder-only, causally masked architectures are specifically suited for, and the reason bidirectional attention was not used.

4.3 Separate Per-Head Weight Matrices:

The Alternative

Rather than a distinct Wq, Wk, Wv per head, a single shared projection could be used across all heads — or, more granularly, only some of Q, K, V could be shared (e.g., unifying K and V while keeping Q separate).

The Trade-off

The original Transformer formulation uses separate per-head projections specifically to let the model jointly attend to information from different representation subspaces at different positions (Vaswani et al., 2017). More recent work shows this comes at a real cost: reducing or sharing projections across heads significantly lowers parameter count and computational overhead, with only minimal impact on downstream performance in many cases — though the effect is task-dependent, with sequential tasks benefiting more from separate, asymmetric projections than non-temporal tasks do. Separately, the memory footprint of the Q/K/V projection activations themselves (not just the weights) has been identified as a substantial, often-overlooked cost at scale.

Why separate Per-Head Weight Matrices is used:

This implementation prioritizes explicit, independently traceable computation for each head over parameter or memory efficiency, following the original formulation (Vaswani et al., 2017). This trade-off

favors clarity here, but would be reconsidered for a production-scale implementation, where the efficiency gains from shared projections become significant.

References

- Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2012.14913
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. *arXiv preprint*. https://arxiv.org/abs/1512.03385
- Karpathy, A. (2023). nanoGPT [Source code]. GitHub. https://github.com/karpathy/nanoGPT
- Kedia, A., Zaidi, M. A., Khyalia, S., Jung, J., Goka, H., & Lee, H. (2024). Transformers get stable: An end-to-end signal propagation theory for language models. *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. *OpenAI*.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research, 21*(140), 1–67.
- Shubham. (2025). Inside sinusoidal position embeddings: A sense of order. *LearnOpenCV*. https://learnopencv.com/sinusoidal-position-embeddings/
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*.
- Wang, T., Roberts, A., Hesslow, D., Le Scao, T., Chung, H. W., Beltagy, I., Launay, J., & Raffel, C. (2022). What language model architecture and pretraining objective works best for zero-shot generalization? *Proceedings of the 39th International Conference on Machine Learning (ICML)*.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. (2020). On layer normalization in the transformer architecture. *Proceedings of the 37th International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2002.04745

What's next:

This document has covered the architecture and forward pass of a from-scratch Transformer implementation: the design decisions behind it, the mathematical formulation of each component, and their corresponding NumPy implementation. The second part of this series will cover the derivation of the backward pass for every component described here, followed by end-to-end training and loss convergence verification on the Tiny Shakespeare dataset.

Since most of the conceptual groundwork and theory has already been established in this document, Part 2 will be considerably more math-heavy, with comparatively little additional theory — the focus

shifts to deriving and presenting the gradient computations themselves, building directly on the architecture and notation defined here.